eGrant Hackathon — zadania

O platforme eGrant

eGrant je SaaS platforma na správu grantových programov. Grantové organizácie (nadácie, mestá, ministerstvá, fondy) v nej riadia celý životný cyklus grantu: manažér programu vytvorí formuláre (žiadosť, hodnotenie, priebežné a záverečné správy, vyúčtovanie), žiadatelia podávajú žiadosti, hodnotitelia ich hodnotia a manažéri generujú dokumenty (zmluvy, potvrdenia) a exporty.

Kľúčové roly:

- **Manažér programu** — nastavuje program, vytvára formuláre a šablóny dokumentov, spracúva žiadosti. Spravidla **nie je technický človek**.
- **Žiadateľ / prijímateľ** — vypĺňa formuláre, nahráva prílohy (faktúry, potvrdenia…), podáva vyúčtovania.
- **Hodnotiteľ** — hodnotí žiadosti cez hodnotiace formuláre.

Čo dostanete

- prístup do testovacej inštancie eGrant (v roli manažéra aj žiadateľa),
- ukážkové formuláre vrátane ich JSON exportov,
- ukážkové šablóny dokumentov (.docx) a k nim vygenerované výsledné dokumenty,
- dokumentáciu tagových funkcií (katalóg funkcií s popismi),
- pre AI zadania prístup MCP

Nebudete mať prístup k zdrojovému kódu platformy. To je zámer: vaše riešenie staviate ako samostatný prototyp v ľubovoľnom stacku. Odovzdať ho môžete napríklad ako React modul alebo ako iný webový modul, ktorý sa dá zapojiť do existujúceho systému. Riešenia, ktoré by vyžadovali prepísať polovicu platformy, neuspejú; riešenia s jasným a úzkym integračným rozhraním áno.

Poznámka: platforma aj dáta sú po slovensky — riešenia (najmä AI zadania) musia dobre fungovať so slovenčinou.

Spoločné hodnotiace kritériá

1. **Použiteľnosť pre netechnického manažéra** — hlavný používateľ nevie programovať.
2. **Integrovateľnosť** — dá sa riešenie reálne zapojiť do existujúceho systému cez úzke, jasne popísané rozhranie?

3. **Funkčný prototyp** — klikateľné demo má prednosť pred prezentáciou.
4. **Pri AI zadaniach: dôveryhodnosť** — presnosť, žiadne vymýšľanie, priznanie „neviem“.

Technický kontext (spoločný pre všetky zadania)

Toto je zjednodušený, ale verný popis toho, ako systém funguje dnes. Detaily si viete pozrieť priamo v testovacej inštancii a v dodaných ukážkach.

Formuláre, položky a tagy

Formulár je **strom**: na najvyššej úrovni sú **sekcie**, v nich položky (a ďalšie podsekcie). Položiek je ~70 typov, v skupinách:

- **kontajnery**: sekcia, podsekcia, opakovateľná podsekcia, popup sekcia, rozcestník (wizard),
- **základné vstupy**: krátky text, dlhý text, WYSIWYG, číslo, suma (mena), e-mail, IBAN, checkbox, select, dátum, obdobie,
- **súbory**: zoznam súborov, zoznam obrázkov,
- **tabuľky**: tabuľka s vlastnou rodinou bunkových typov (text, číslo, suma, dátum, select, súbor, poradové číslo riadku, počítané bunky…),
- **rozpočet**: špecializované rozpočtové sekcie,
- **počítané polia**: súčet, súčin, percento,
- **pokročilé**: skryté polia, JS kód, validačné položky, generátor PDF, import z Excelu…

Najdôležitejší koncept celého systému je **tag**. Každá položka formulára má unikátny tag (napr. `projekt.nazov`, `ziadatel.ico`, `rozpocet`). Hodnoty vyplnené žiadateľom sa ukladajú **pod tagom**, nie pod položkou — tag je „adresa“ údaja. Na tagy sa odkazuje všetko ostatné: podmienky zobrazenia, výpočty, šablóny dokumentov, exporty, notifikácie. Tag je teda kontrakt, ktorý sa nesmie stratiť pri žiadnom redizajne.

Výrazy — tagové funkcie

Podmienky a výpočty sa píšu vo vlastnom výrazovom jazyku. Sú to **iba prefixové funkcie** (žiadne operátory ako `==` alebo `>`), argumenty sú tagy, literály v úvodzovkách alebo vnorené funkcie:

```
    gt(projekt.suma, 5000)                        — suma je väčšia ako
5000
    and(eq(typ_ziadatela, "OZ"), not(empty(ico))) — kombinácia
podmienok
    sum(rozpocet_osobne, rozpocet_materialove)    — súčet dvoch polí
    if(gt(projekt.suma, 5000), "áno", "nie")      — podmienená hodnota
    in_array(time_year_array(prj.zaciatok, prj.koniec), 2027) — projekt
zasahuje do roku 2027
```

K dispozícii je ~110 funkcií: logika a porovnania (`eq`, `gt`, `and`, `or`, `if`, `coalesce`, `empty`…), matematika (`sum`, `round`, `percent`, `number_format`, `currency`, `spell` — suma slovom…), reťazce (`concat`, `uppercase`, `replace`…), dátumy (`date`, `now`…), práca s poľami/tabuľkami (`array_map`, `array_filter`, `array_sum`, `count`…) a prístup k dátam (`tag`, `active_user`, hodnoty z iných formulárov…). Dostanete kompletný katalóg.

Každá položka formulára môže mať výrazy pre: podmienku **zobrazenia**, podmienku **vyplniteľnosti**, podmienku **validácie**, podmienenú **povinnosť**, **spracovanie dát** a **zamknutie editácie**. Zďaleka najpoužívanejšia je podmienka zobrazenia („zobraz túto sekciu, len ak…“).

Formulár ako JSON

Celý formulár (alebo jeho časť) sa dá exportovať a importovať ako JSON. Toto je existujúca, plne funkčná cesta, ako dostať do systému formulár „zvonku“. Ilustračná, skrátená ukážka (reálne exporty dostanete):

```
    {
      "type": "root",
      "children": [
        {
          "type": "SECTION",
          "name": "Základné údaje o projekte",
          "tag": "zakladne_udaje",
          "children": [
            { "type": "SIMPLE_STRING", "name": "Názov projektu", "tag":
"projekt.nazov", "mandatory": true },
            { "type": "CURRENCY", "name": "Požadovaná suma", "tag":
"projekt.suma", "mandatory": true },
            {
              "type": "TEXT",
              "name": "Zdôvodnenie vyššej sumy",
              "tag": "projekt.zdovodnenie",
              "view_condition": "gt(projekt.suma, 5000)"
            },
            {
              "type": "TABLE_LIST",
              "name": "Rozpočet",
              "tag": "rozpocet",
```

```
            "children": [
              { "type": "TABLE_ITEM_STRING", "name": "Položka",
"tag": "polozka" },
              { "type": "TABLE_ITEM_CURRENCY", "name": "Suma", "tag":
"suma" }
            ]
          }
        ]
      }
    ]
  }
```

Pravidlá v skratke: na najvyššej úrovni smú byť len sekcie; typy položiek majú povolené vnorenia (napr. bunky tabuľky smú byť len v tabuľke); typová hodnota `type` je názov typu; typovo špecifické nastavenia (napr. možnosti selectu) žijú v kľúči `config`.

Šablóny dokumentov

Manažér nahrá k programu **šablónu vo Worde** (.docx) a systém z nej pre konkrétnu žiadosť vygeneruje dokument (Word, PDF alebo ODT). Šablóna je obyčajný wordovský dokument, v ktorom sú zástupné značky v zložených zátvorkách:

```
Žiadateľ: {ziadatel.nazov}, IČO: {ziadatel.ico}
Dátum podania: {date("d.m.Y", application.submitted_at)}
Požadovaná suma: {number_format(projekt.suma, 2, ",", " ")} EUR

{IF:gt(projekt.suma, 5000)}
Žiadosť podlieha schváleniu správnej rady.
{ELSE:gt(projekt.suma, 5000)}
Žiadosť schvaľuje riaditeľ.
{ENDIF:gt(projekt.suma, 5000)}
```

- `{tag}` sa nahradí hodnotou; vnútri zátvoriek sa dajú použiť **tie isté tagové funkcie** ako vo formulároch (formátovanie dátumov, súm, podmienené hodnoty, suma slovom…).
- Podmienené bloky `{IF:výraz}` … `{ELSE:výraz}` … `{ENDIF:výraz}` zobrazia/skryjú časť dokumentu.
- **Tabuľky**: do prvej bunky riadku wordovskej tabuľky, ktorý sa má opakovať, sa vloží značka `{TABLE:rozpocet}`; do buniek riadku sa píšu značky stĺpcov (`{polozka}`, `{suma}`, `{_index}` pre poradové číslo). Systém riadok naklonuje pre každý riadok dát. Existuje aj `{SUBTABLE:…}` pre jedno vnorenie a variant opakovania bloku viacerých riadkov.
- Neznáme alebo prázdne tagy sa potichu nahradia prázdnym reťazcom.

Prečo je to problém v praxi: šablóna sa píše ručne vo Worde — bez našepkávania, bez náhľadu, bez kontroly preklepov. Chyba sa prejaví až pri generovaní (alebo horšie — nikto si ju nevšimne a v zmluve chýba údaj). Word navyše pri editovaní rád „rozbije“ značku na viac vnútorných častí, takže značka vyzerá správne, ale systém ju nenájde. Najbolestivejšie sú tabuľky: presná syntax, značka musí byť v správnej bunke, opakovanie viacriadkových blokov je pozičné. Výsledok: šablóny dnes často nevyrába manažér, ale naša podpora.

Zadanie 1 — AI extrakcia údajov z nahraných súborov

Kontext

Pri vyúčtovaní grantu prijímatelia nahrávajú do formulára doklady — faktúry, bločky, zmluvy, výpisy z účtu — a **zároveň ručne prepisujú ich údaje** do tabuliek formulára: číslo faktúry, dodávateľ, IČO, dátumy, sumy, jednotlivé položky. Prepisovanie je zdĺhavé a chybové a manažér potom ručne kontroluje, či sa vyplnené údaje zhodujú s prílohou.

Úloha

Navrhnite a prototypujte funkciu: používateľ nahrá súbor a AI z neho vytiahne štruktúrované údaje a **predvyplní príslušné polia formulára**. Používateľ výsledok skontroluje, opraví a potvrdí — nič sa nezapíše bez jeho kontroly.

Na čo myslieť:

- **Rôzne vstupy**: čisté PDF, sken, fotka z mobilu, Excel. (V systéme už existuje OCR služba, ktorá prevedie PDF/sken na text — dostub k nej pomocou API dostanete.
- **Štruktúrovaný výstup**: pre faktúru napr. číslo faktúry, dodávateľ, IČO/DIČ, dátum vystavenia a splatnosti, suma s DPH a bez, jednotlivé položky. Formuláre sú však rôzne — extrakcia by nemala byť natvrdo „len faktúra“.
- **Mapovanie na formulár**: extrahované údaje treba priradiť poliam formulára (tagom) — ideálne tak, aby si mapovanie vedel nakonfigurovať manažér pre svoj formulár sám (napr. „z faktúry vyplň `cislo_faktury`, `dodavatel.ico`, riadky do tabuľky `polozky`“).
- **Dôveryhodnosť**: ak údaj v doklade nie je, AI ho **nesmie vymyslieť** — pole ostane prázdne. Užitočné je signalizovať istotu (ktoré polia si vyžadujú kontrolu).
- **UX potvrdenia**: ako používateľ uvidí, čo sa predvyplnilo, a ako to opraví.

Očakávaný výstup

- Funkčný prototyp: nahranie súboru → extrakcia → predvyplnený formulár (stačí vlastná napodobenina formulára).
- Návrh integrácie: ako manažér nakonfiguruje extrakciu pre svoj formulár, kadiaľ tečú dáta, aké rozhranie by platforma volala.

Hodnotí sa

Presnosť extrakcie na dodaných vzorkách (vrátane škaredých skenov), správanie pri chýbajúcich údajoch, realistickosť integrácie.

Zadanie 2 — AI asistent nad grantovými programami + generovanie formulárov

Kontext

Manažéri aj žiadatelia sa neustále pýtajú na to, čo už v systéme je: aké sú uzávierky, aké prílohy treba, čo obsahuje formulár, aké sú podmienky programu. Odpovede existujú — v nastaveniach programov, formulároch, textoch výziev, šablónach — len sú roztrúsené a treba ich hľadať.

Úloha

Postavte asistenta v štýle NotebookLM nad dátami grantových programov, s dvomi schopnosťami:

A) Odpovedanie na otázky — výhradne z dát systému.

- Asistent odpovedá **len** na základe dodaných dát (programy, výzvy, formuláre, texty, termíny). Toto je tvrdá požiadavka číslo jeden: **žiadne vymýšľanie**.
- Pri každej odpovedi vie ukázať **zdroj** (z ktorého programu/formulára/textu odpoveď pochádza).
- Keď odpoveď v dátach nie je, povie, že nevie — nedopĺňa ju zo všeobecných znalostí o grantoch.
- Otázky aj odpovede po slovensky.

B) Generovanie celých grantových formulárov.

- Na požiadanie („vytvor formulár žiadosti pre program podpory kultúry: údaje o žiadateľovi, popis projektu, rozpočtová tabuľka, povinné prílohy…“) asistent vygeneruje **kompletný formular.**
- Asistent by mal vedieť formulár aj iteratívne upravovať („pridaj sekciu o udržateľnosti“, „sprav IČO povinným“).

Očakávaný výstup

- Funkčný chat prototyp
- Generovanie formulárov s validáciou výstupu proti pravidlám štruktúry.
- Popis architektúry: ako zabezpečujete, že asistent nevymýšľa a ako by ste to merali.

Hodnotí sa

Odolnosť proti vymýšľaniu, kvalita a dohľadateľnosť citácií, štrukturálna platnosť a zmysluplnosť vygenerovaných formulárov, UX.

Zadanie 3 — Redizajn form buildera

Kontext

Formuláre dnes manažér skladá v builderi: vľavo paleta ~70 typov položiek, v strede strom formulára (drag & drop), po kliknutí na položku editačný panel s desiatkami nastavení — názov, tag, povinnosť, šesť polí na výrazy, vlastné šablóny zobrazenia, správy validácií, prístupové práva… K tomu existujú „power“ funkcie: tagové výrazy, vlastné Handlebars šablóny položiek, JavaScript na úrovni formulára, pokročilé typy položiek (JS kód, validačné položky, skryté počítané polia).

Realita: bežný manažér používa zlomok z toho a z výrazov a kódov má strach — no zároveň sú to práve podmienky zobrazenia, ktoré netechnickí manažéri reálne potrebujú (sú najpoužívanejším výrazovým poľom), akurát ich dnes musia písať ako `gt(projekt.suma, 5000)`. Formuláre pritom nie sú malé: bežný má desiatky položiek, veľké stovky až tisícky.

Úloha

Navrhnite **nový spôsob tvorby formulárov** — nie kozmetiku existujúceho UI, ale premyslený koncept, ktorý:

- **zachová všetky súčasné koncepty** — predovšetkým **tagy** (sú kontraktom pre celý zvyšok systému — šablóny, exporty, podmienky), hierarchiu sekcií a položiek, typy položiek, povinnosť, podmienky;
- **rozhodne, čo schovať**: ktoré funkcie patria do „advanced“ režimu (výrazy, custom šablóny, kódy, pokročilé typy) a ako sa k nim power user dostane bez toho, aby zavadzali bežnému manažérovi;
- **sprístupní podmienky bez písania výrazov** — napr. vizuálny editor podmienok („zobraz, keď [Požadovaná suma] [je väčšia ako] [5000]“), ktorý sa na pozadí uloží ako existujúci výraz;
- premyslí prácu s tagmi (dnes ich manažér vymýšľa ručne; kedy ich generovať automaticky a kedy musia byť pod kontrolou?), náhľad formulára očami žiadateľa a prácu s veľkými formulármi.

Očakávaný výstup

- Interaktívny prototyp (klikateľný HTML mock alebo Figma) hlavných tokov: vytvorenie formulára, pridanie sekcie a položiek, nastavenie podmienky zobrazenia, tabuľka/rozpočet, prepnutie do advanced režimu.
- **Mapa konceptov**: každý súčasný koncept → kde žije v novom UI (a čo je skryté kde). Nič sa nesmie stratiť — len presunúť.
- Krátke zdôvodnenie rozhodnutí.

Hodnotí sa

Zvládne netechnický manažér scenár „vytvor formulár žiadosti s podmienenou sekciou a rozpočtovou tabuľkou“ bez manuálu? Ostala zachovaná plná sila systému pre power userov? Je návrh kompatibilný s existujúcim dátovým modelom (formulár ako strom položiek s tagmi — JSON export musí byť z nového buildera stále vyrobiteľný)?

Zadanie 4 — Vizuálny builder šablón dokumentov

Kontext

Dokumenty (zmluvy, potvrdenia) sa generujú z wordovských šablón so značkami `{tag}`, tagovými funkciami, podmienenými blokmi `{IF:…}` a tabuľkovými značkami `{TABLE:…}`. Písanie šablón vo Worde je krehké a bez spätnej väzby — a najviac to bolí pri tabuľkách. V praxi šablóny často nevyrábajú manažéri, ale naša podpora.

Úloha

Navrhnite a prototypujte **vizuálny builder šablón dokumentov**, v ktorom šablónu zvládne vytvoriť netechnický manažér:

- **Editor dokumentu** s formátovaním (nadpisy, tučné, zoznamy…) — dokument má vyzerať ako výsledná zmluva.
- **Vkladanie polí výberom**, nie písaním: picker nad štruktúrou formulára (dodáme JSON exporty formulárov) namiesto ručného vypisovania `{ziadatel.ico}`.
- **Formátovanie cez UI**: dátum, mena, suma slovom… ako voľba pri poli, nie ako ručne písaná funkcia `{number_format(…)}`.
- **Podmienené časti** cez vizuálny editor podmienok (rovnaká myšlienka ako v zadaní 3).
- **Tabuľky** — kľúčová časť: návrh tabuľky naviazaný na tabuľkovú položku formulára (výber stĺpcov, poradové číslo, súčtový riadok), bez ručných značiek.
- **Náhľad** so skutočnými dátami ukážkovej žiadosti — manažér vidí výsledok pred uložením.

Integračná cesta (dôležité): najjednoduchšia a preferovaná možnosť je, aby builder na výstupe **vygeneroval klasickú .docx šablónu s existujúcou syntaxou značiek** — tú potom systém spracuje ako doteraz, bez zásahu do generátora.

Očakávaný výstup

- Funkčný prototyp buildera (aspoň: text + polia + formátovanie hodnôt + podmienený blok + tabuľka + náhľad).
- Vygenerovaná šablóna — ideálne .docx so správnou syntaxou značiek; správnosť overíme na existujúcom generátore (dodáme vzorové šablóny a k nim vygenerované dokumenty na porovnanie).
- Návrh integrácie a úprimný zoznam toho, čo builder nepokrýva (a ako by manažér riešil výnimky — napr. „otvor ako Word šablónu“).

Hodnotí sa

Zvládne šablónu vytvoriť netechnický manažér? Tabuľky, tabuľky, tabuľky. Vernosť náhľadu voči reálnemu výstupu. Kompatibilita vygenerovanej šablóny s existujúcim generátorom. Rozumný „únikový východ“ pre prípady, ktoré builder nepokryje.